Фиксированный размер целочисленных типов данных

На уроке о **целочисленных типах данных** мы говорили, что C++ гарантирует только их минимальный размер — они могут занимать и больше, в зависимости от компилятора и/или архитектуры компьютера.

Оглавление:

1. Почему размер целочисленных типов не является фиксированным?
2. Разве это не глупо?
3. Целочисленные типы фиксированного размера
4. Предупреждение насчет std::int8_t и std::uint8_t
5. Недостатки целочисленных типов фиксированного размера
6. Спор насчет unsigned

Почему размер целочисленных типов не является фиксированным?

Если говорить в общем, то всё еще началось с языка Си, когда производительность имела первостепенное значение. В языке Си намеренно оставили размер целочисленных типов нефиксированным для того, чтобы компилятор мог самостоятельно подобрать наиболее подходящий размер для определенного типа данных в зависимости от компьютерной архитектуры.

Разве это не глупо?

Возможно. Программистам не всегда удобно иметь дело с переменными, размер которых варьируется в зависимости от компьютерной архитектуры.

Целочисленные типы фиксированного размера

Чтобы решить вопрос кроссплатформенности, в язык C++ добавили набор **целочисленных типов фиксированного размера**, которые гарантированно имеют один и тот же размер на любой архитектуре:

Название	Тип	Диапазон значений
int8_t	1 байт signed	от -128 до 127
uint8_t	1 байт unsigned	от 0 до 255
int16_t	2 байта signed	от -32 768 до 32 767
uint16_t	2 байта unsigned	от 0 до 65 535
int32_t	4 байта signed	от -2 147 483 648 до 2 147 483 647
uint32_t	4 байта unsigned	от 0 до 4 294 967 295
int64_t	8 байт signed	от -9 223 372 036 854 775 808 до 9 223 372 036 854 775 807
uint64_t	8 байт unsigned	от 0 до 18 446 744 073 709 551 615

Начиная с C++11 доступ к этим типам осуществляется через подключение **заголовочного файла** cstdint (находятся эти типы данных в **пространстве имен std**). Рассмотрим пример на практике:

```
#include <iostream>
#include <cstdint>

int main()
{
    std::int16_t i(5); // прямая инициализация
    std::cout << i << std::endl;
    return 0;
}
```

Поскольку целочисленные типы фиксированного размера были добавлены еще до C++11, то некоторые старые компиляторы предоставляют доступ к ним через подключение заголовочного файла stdint.h.

Если ваш компилятор не поддерживает cstdint или stdint.h, то вы можете скачать кроссплатформенный заголовочный файл **pstdint.h**. Просто **подключите его** к вашему проекту, и он самостоятельно определит целочисленные типы фиксированного размера для вашей системы/архитектуры.

Предупреждение насчет std::int8_t и std::uint8_t

По определенным причинам в C++ большинство компиляторов определяют и обрабатывают типы int8_t и uint8_t идентично типам char signed и char unsigned (соответственно), но это происходит далеко не во всех случаях. Следовательно, **std::cin и std::cout** могут работать не так, как вы ожидаете. Например:

```
#include <iostream>
#include <cstdint>

int main()
{
    std::int8_t myint = 65;
    std::cout << myint << std::endl;

    return 0;
}
```

На большинстве компьютеров с различными архитектурами результат выполнения этой программы следующий:

```
A
```

Т.е. программа, приведенная выше, обрабатывает `myint` как переменную типа char. Однако на некоторых компьютерах результат может быть следующим:

```
65
```

Поэтому идеальным вариантом будет избегать использования std::int8_t и std::uint8_t вообще (используйте вместо них std::int16_t или std::uint16_t). Однако, если вы все же используете std::int8_t или std::uint8_t, то будьте осторожны с любой функцией, которая может интерпретировать std::int8_t или std::uint8_t как символьный тип, вместо целочисленного (например, с объектами std::cin и std::cout).

Правило: Избегайте использования std::int8_t и std::uint8_t. Если вы используете эти типы, то будьте внимательны, так как в некоторых случаях они могут быть обработаны как тип char.

Недостатки целочисленных типов фиксированного размера

Целочисленные типы фиксированного размера могут не поддерживаться на определенных архитектурах (где они не имеют возможности быть представлены). Также эти типы могут быть менее производительными, чем фундаментальные типы данных, на определенных архитектурах.

Спор насчет unsigned

Многие разработчики (и даже большие организации) считают, что программисты должны избегать использования целочисленных типов unsigned вообще. Главная причина — непредсказуемое поведение и результаты, которые могут возникнуть при «смешивании» целочисленных типов signed и unsigned в программе.

Рассмотрим следующий фрагмент кода:

```
void doSomething(unsigned int x)
{
    // Выполнение некоего кода x раз
}

int main()
{
    doSomething(-1);
}
```

Что произойдет в этом случае? `-1` преобразуется в другое большое число (скорее всего в `4 294 967 295`). Но самое грустное в этом то, что предотвратить это мы не сможем. Язык C++ свободно конвертирует числа с типами unsigned в типы signed и наоборот без проверки диапазона допустимых значений определенного типа данных. А это, в свою очередь, может привести к переполнению.

Бьёрн Страуструп, создатель языка C++, считает, что: «Использовать тип unsigned (вместо signed) для получения еще одного бита для представления положительных целых чисел, почти никогда не является хорошей идеей».

Это не означает, что вы должны избегать использования типов unsigned вообще — нет. Но если вы их используете, то используйте только там, где это действительно имеет смысл, а также позаботьтесь о том, чтобы не допустить «смешивания» типов unsigned с типами signed (как в вышеприведенном примере).